

# Fit for the Engine? Image-to-3D Meshes, Their Pipeline, and an AI Assistant’s Explanations in a Completed Unity Game

Penggan Zhao  
Independent Researcher  
China  
pgzhao101@gmail.com

## Abstract

A solo developer built a complete Unity game with an AI coding assistant and filled it with image-to-3D models. We audited its 26 generated meshes against a plan committed before any audit script ran, and then did something the usual evaluation of generated 3D cannot: we set the assistant’s own written explanations of the asset problems, kept in its commit messages, tool comments and design notes, against independent measurements of the same assets. The raw meshes are closed and edge-manifold, and their UV layouts resemble the game’s purchased meshes; most problems came from the pipeline around the generator. The importer reversed the characters’ front (glTF puts it at +Z); the import path stored each asset’s 4096<sup>2</sup> base map uncompressed and dropped its normal and metallic-roughness maps, so 24 maps make up 70 % of the build’s texture data; MeshLab simplification of the seam-split buffer tore the companion into 2,433 pieces, where welding first leaves every asset intact. Of 39 testable claims the assistant made about the assets, 8 are wrong; the confirmed ones are mostly readings off a file, the wrong ones mostly explanations: five blame the generator for what the pipeline did. Five of the six contradicted claims were never revised, and one spread from tool to tool as a “finding”. Every defect clearly visible in play was fixed within an hour of being reported; every defect not visible on screen shipped, including one an AI review had flagged. We release scripts, measurements, the claims with both judges’ verdicts, the defect record and the review reports.

## Keywords

image-to-3D generation, game assets, asset pipelines, AI coding assistants, root-cause analysis, mesh simplification, content audit

## 1 Introduction

Image-to-3D generators now turn a single drawing into a textured mesh in a minute or two [40, 46], and AI coding assistants write much of the code that imports, converts and places those meshes. A solo developer can now build a complete game this way. The game studied here, ChronoTraveler, has 15 non-player characters and nine props that each came from one drawing, and 410 of its 484 commits were co-authored by an assistant.

Generated meshes are usually judged by how they look. Engine readiness depends on other properties: size and orientation, origin, closed and manifold surfaces, memory, whether the mesh survives simplification, and, once rigged, where the feet are when it moves. When something goes wrong, a second question follows: whose fault is it, the generator’s or the pipeline’s? In an assistant-built game that question is usually answered by the assistant, in a commit

message or a code comment, and the answer then shapes the next fix.

This paper audits the generated meshes of the completed game against those properties, and then audits the assistant’s explanations. We committed an analysis plan before writing any audit script, measured every generated asset, compared them with the game’s purchased meshes, read the textures the shipped build actually stores, simplified each mesh under several inputs and tools, and evaluated the rigged boss’s animations frame by frame. A second plan, committed before any extraction, set out how to collect every claim the assistant made about the assets and judge it against those measurements, and how to reconstruct each defect’s history from git. Internal fact-checks and a review of the first draft led to further measurements, which we mark as such.

Most of what went wrong came from the pipeline, not the generator, and several of the assistant’s explanations said the opposite. Its wrong claims were mostly explanations rather than readings; most blamed the generator; they were written into tool comments, copied to other tools and never revised. Defects that showed on screen were fixed within the hour; defects that did not, from uncompressed textures to a torn mesh, shipped.

Contributions: (1) an audit of every generated mesh in a completed game, with scripts, measurements and review reports released; (2) an attribution audit of an AI coding assistant’s contemporaneous explanations of generated-asset problems against measured ground truth, which to our knowledge has not been done; (3) a defect record showing which problems a visual play-test catches and which it does not; (4) a pre-import checklist derived from the case and not yet validated elsewhere. It extends our audits of AI-generated game content [44, 45] from text to geometry and to the assistant that builds the pipeline.

## 2 Related Work

*Image-to-3D generators.* Feed-forward and diffusion systems produce textured meshes from one image in seconds to minutes: TripoSR [31], InstantMesh [41], SF3D [4], TRELLIS [40] and the Hunyuan3D series [27, 46], whose commercial service made our assets [28]. Several properties we measure follow from such designs. Hunyuan3D 2.1 normalises training shapes to a unit cube and extracts surfaces as isosurfaces [27], which predicts unit scale and closed meshes; its open-source texturing defaults to 4096<sup>2</sup> maps and unwraps with default xatlas settings [29], and its developers note that xatlas “generates over-fragmented cuts” [19]. We cannot confirm that the commercial service runs the same code.

*Generated assets in production.* Evaluation of generated 3D is mostly image-based: T3Bench [12], GPTEval3D [39] and 3DGenBench [43] score rendered views. Work on artist-like topology [5,

26] changes the output representation instead. A 2026 survey of production-ready generation observes that packing efficiency, manifoldness and watertightness are “rarely reported” or “almost never quantified” [38]. PartUV measures chart counts on 114 TRELLIS meshes (xatlas: median 895, mean 1,542) [37]. The closest study to ours is Begemann and Hutson’s practice-led comparison, which rebuilt one game scene with Meshy and Hunyuan 3D and reported fragmented UV maps and loss of texture integrity under decimation [2]. Practitioners describe generated UVs as “a jumbled mess” [8], and HCI studies document how solo and indie developers use generative tools [24]. Our study differs in measuring every generated asset of a completed game, per asset, against stated hypotheses, and in tracing each problem to where in the pipeline it arose.

*Conventions and import.* glTF fixes units (metres), up (+Y) and front (+Z), and requires a separate vertex wherever an attribute is discontinuous [30]; Khronos’s importer for Blender offers to merge such vertices only as an option [17]. Khronos’s real-time asset guidelines ask for 1:1 real-world scale and an origin at the bottom centre [16], and engine vendors note that textures usually dominate memory [32]. Several items of our checklist therefore restate existing guidance; what this case adds is which violations occurred, where they came from and which ones went unnoticed.

*Simplification with seams.* Quadric error simplification [10] was extended to appearance attributes [11, 14]; Hoppe’s wedge structure stores one vertex with several attribute values at a seam [14]. Texture-aware simplification bounds texture stretch [25], seam-aware methods preserve the parametrisation exactly [21], and recent work simplifies textured meshes independently of their UV layout [20]. meshoptimizer’s documentation states that “identical vertices” must be welded before simplification [15]; PyMeshLab’s documentation says its per-wedge conversion “does not merge superfluous vertices” [36]; a simplifier evaluated on Hunyuan3D output merges coincident vertices as standard pre-processing [3]; and an open-source game project hit the same cracking on Meshy assets and also welded first (then re-unwrapped and re-baked), two weeks before we wrote this [9]. The mechanism is known; we report how much damage it did in a shipped asset and that the filter gave no sign of it.

*Grounding animated characters.* Keeping feet on the ground is a per-frame problem, handled by foot-contact cleanup or inverse kinematics [18]. Unity offers a per-frame “Feet” root mode that “prevents [the] floating problem”, but only for Humanoid rigs [35]; our boss is a Generic rig. Unity precomputes a skinned mesh’s bounds from its animations unless asked to update them every frame [33], and its pose snapshot API has a scale option whose effect depends on the hierarchy [34].

### 3 The Game and Its Generated Assets

ChronoTraveler is a turn-based educational RPG built in Unity 6 (URP) by one developer working with an AI coding assistant in mid-2026 (repository history June–August), as an MSc project. We call its final build (game commit 87fdd16) the *completed* game; it was not released commercially. Its five historical eras each have three non-player characters (NPCs); the Silk Road era also has

**Table 1: Audited meshes. “Raw” = generator output as downloaded.**

| Group               | <i>n</i> | Source and use                                            |
|---------------------|----------|-----------------------------------------------------------|
| NPCs                | 15       | image-to-3D, 50k setting; raw GLB, static                 |
| Props               | 9        | same pipeline; raw GLB, static                            |
| Companion (raw)     | 1        | image-to-3D, high setting; 497,850 triangles; not shipped |
| Companion (shipped) | 1        | our simplification of the raw mesh; 82,829 triangles      |
| Boss                | 1        | studio pipeline: retopology, rig, 4 animations (4 FBX)    |

nine props (two camels, a horse, a cart, a stupa, a milestone, silk bolts and spice sacks). All 24 were generated from one front-view drawing each with Tencent’s Hunyuan 3D web service (image-to-3D, version 3.1 according to the game’s pipeline notes, “50k faces” setting) and downloaded as GLB. Two more meshes came from the same service: the companion fox, generated at a higher setting (497,850 triangles) and simplified by us to 82,829 triangles before shipping, and the final boss, produced by the service’s studio pipeline [19] with retopology, UV unwrapping, texturing, a 28-bone rig and four animations, delivered as four FBX files that share one 9,038-triangle mesh. We audit 26 meshes, of which 25 shipped (Table 1); `data/manifest.json` lists every file with its SHA-256.

The 24 GLBs in the game are byte-identical to the downloads. They were imported with UniGLTF 0.131 (the glTF importer bundled with UniVRM), whose default converts glTF to Unity coordinates by reversing Z. An editor tool then rescaled each NPC to 1.7 m, turned it 180° and replaced its material with URP/Lit; a second tool does the same for the boss at 10 m; a run-time component (GroundSnap) moves the animated boss down onto its platform.

### 4 Method

The audit plan—metrics, five hypotheses and decision rules—was committed and pushed to the study repository (commit 2b0bc54, 16:32 on the day of the audit) before any audit script existed. It is not a registered report: the plan was written by the same person who built the game and wrote the commits that B5 labels, so B5 and E1 are not blind. Two of the five hypotheses restate facts a read-only survey had already seen while scoping the study (triangle counts, texture sizes, the double-sided flag, the root rotation, unit-scale heights); the plan says so. The scripts and all first results were then committed together 21 minutes later, so the order in which they ran is not separately recorded. Everything added after the first draft, in response to an internal fact-check and review, is marked as such and listed in Section 8.

*Terms.* glTF stores a *seam-split* vertex buffer: wherever a per-vertex attribute such as a UV coordinate is discontinuous, the vertex is duplicated. *Welding* merges vertices with identical positions and keeps UVs per corner, as *wedge* attributes [14]. After welding, a mesh is *closed* if it has no boundary edges and *edge-manifold* if every edge has exactly two incident triangles. MiB is 2<sup>20</sup> bytes.

*B1 Conventions.* A small GLB reader (no glTF library, so the numbers come from the bytes any importer reads) applies each file’s

node chain and records the bounding box, the largest extent, the origin as a fraction of the box per axis, node transforms, material flags and extensions, and each embedded image’s format and size. **H1**: every raw asset is scale-normalised (largest extent 0.9–1.2 units), carries a non-identity root rotation and is double-sided. *Added after review*: each GLB was rendered in glTF coordinates from +Z and –Z to see which side is the front.

**B2 Hygiene**. After welding: connected components (triangles sharing a vertex), boundary and non-manifold edges, non-manifold vertices, degenerate and duplicate triangles, and self-intersecting triangles (MeshLab [6] via PyMeshLab [23]). Before welding: UV charts (seams separate them) and UV coverage, the share of the unit square covered by UV triangles rasterised at  $2048^2$ . Rasterisation counts edge texels, so coverage is an upper bound by about two points; chart overlap was not measured. **H2**: a majority of the 24 raw assets fail at least one of closed, edge-manifold, or single component. *Added after review*: the same measurements on the game’s 81 non-generated meshes (a licensed character and three purchased environment packs).

**B3 Cost**. Geometry bytes as stored and texture bytes for three GPU formats with full mip chains: RGBA8 (4 B/texel), BC7 (1 B/texel), BC1 (0.5 B/texel). **H3**: texture memory exceeds geometry memory at least tenfold for every raw asset in every format. *Added after review*: the textures actually stored in the shipped Windows build, read with UnityPy.

**B4 Simplification**. Texture-aware quadric edge collapse [10, 11, 14] as implemented in MeshLab (quality threshold 0.3, boundary preservation off) at 50, 25, 12.5 and 6.25 % of the triangles, up to three passes. The plan named a with-UV and a no-UV condition. Because B2 found the shipped companion torn, the with-UV condition was run on two inputs: *S*, the seam-split buffer that MeshLab’s glTF loader returns (what our companion pipeline used), and *W*, the same file after `meshing_remove_duplicate_vertices`. *N* is welded, no UVs. We record triangles reached, boundary edges in the output after welding, and surface distance to the original sampled Metro-style [7] (200,000 samples per direction; the larger of the two directions, as a percentage of the bounding-box diagonal; we report the mean and the maximum). **H4**: with UVs preserved, most assets cannot reach 6.25 % within three passes while the no-UV control can. *Added after review* (12.5 and 6.25 %): the same *S* and *W* inputs in meshoptimizer [15]; *S* with boundary preservation on; *N* with the same normal flag as *S* and *W*; and a colour error for *S* and *W*, comparing base colour looked up through the simplified UVs with base colour at the nearest original vertex (60,000 samples; the same procedure on the unsimplified mesh gives the noise floor).

**B5 History**. All 484 non-merge commits up to the final build were searched with a pre-stated pattern of 25 terms (scale, pivot, ground, clip, cull, bounds, glb, decimat, ...). Each of the 156 matches was labelled ASSET (root cause is a property of a generated asset), ENGINE (an engine or tool behaviour any asset would hit), LEVEL (level geometry, placement, purchased or procedural content) or OTHER, with ambiguous cases sent to OTHER. Two AI labellers drafted the labels, each taking half; the AI assistant reviewed every ASSET, ENGINE and low-confidence label in two rounds, recording each change with its reason, and the author signed off. For ASSET commits we

note which B1–B3 measurement, if any, would have shown the property before import. **H5**: most ASSET fixes concern properties such counting would reveal.

**E1 Grounding (exploratory)**. In Blender 5.1 we evaluate the boss’s skinned mesh at every frame of each animation (60 fps) and take the lowest vertex as ground truth. Heights are scaled so that the bind pose is 10 m tall, as in the game. We score the estimators the game tried: bind-pose bounds, one frame, and the shipped minimum over ten samples 0.16 s apart, at every starting phase of the loop.

**C Attribution audit (second plan)**. After the first results, a second plan was committed (63b4a34) before any claim was extracted. 410 of the 484 commits carry the AI assistant’s co-author trailer; their messages, the comments in the asset tools and the design notes explain what the generated assets are and why problems occurred. The source texts were frozen (174 commits, 215 comment lines, five design notes). Two AI extractors, without access to the measurements, independently listed every *claim*: a declarative statement about a property of a generated asset, about what the pipeline does to it, or about the cause of an asset problem. The merged claims were judged by the assistant and, independently, by a second AI judge as CONFIRMED, CONTRADICTED, PARTLY (right observation, wrong cause or a magnitude off by more than  $2\times$ ) or UNTESTABLE, using only this study’s measurements and two measurements listed in the plan in advance (the origin after UniGLTF’s import; what MeshLab’s loader does with the root node). Disagreements were resolved under a rule written before resolving; the author signed off. **H6**: at least 20 % of testable claims are contradicted or partly wrong. **H7**: most contradicted claims are never revised before the final build. **H8** (exploratory): causal claims are wrong more often than property claims.

**D Defect lifecycle**. For every defect the audit identified and every ASSET/ENGINE fix, we reconstructed from git when it was introduced, first described (and by whom: play-test, import inspection, the assistant, an AI review, this audit), fixed, and whether it reached the final build. **H9** (descriptive): defects visible on screen were fixed during development; defects not visible on screen shipped. The interactive session transcripts of the development period no longer exist (the earliest surviving one is from September), so effort is measured in commits and elapsed time.

## 5 Results

Table 2 summarises the hypotheses. Of the first five, the two that were met, or nearly met, restate what the scoping survey had already seen; none of the three that had not been looked at held as first analysed.

### 5.1 Conventions: size from outside, front by the importer

All 24 raw assets carry the same root rotation ( $+90^\circ$  about *X*, so the mesh is stored with its up axis along local –Z), all are double-sided, and all use the same material: one primitive, one material with `KHR_materials_specular`, and three  $4096^2$  PNG maps (base colour, normal, metallic-roughness). Every file names Blender’s glTF exporter as its generator. Scale is normalised whatever the subject: the largest extent is 0.895–1.151 units (median 1.082), and

**Table 2: Hypotheses as stated in the two plans and their outcome.**

|    | Statement (abridged)                                                  | Outcome                          |
|----|-----------------------------------------------------------------------|----------------------------------|
| H1 | all raw assets scale-normalised (0.9–1.2), rotated root, double-sided | not met (23/24)                  |
| H2 | most raw assets not closed, not manifold, or several components       | not supported (5/24)             |
| H3 | texture $\geq 10\times$ geometry memory, all assets, all formats      | supported (24/24)                |
| H4 | with UVs, most assets cannot reach 6.25 %                             | not supported (S: 6/24, W: 1/24) |
| H5 | most ASSET fixes visible to counting                                  | supported only after relabelling |
| H6 | $\geq 20\%$ of testable assistant claims wrong                        | supported (8/39)                 |
| H7 | most contradicted claims never revised                                | supported (5/6)                  |
| H8 | causal claims wrong more often (expl.)                                | 2/5 vs 3/14 (too few)            |
| H9 | visible defects fixed, invisible ones ship                            | supported                        |

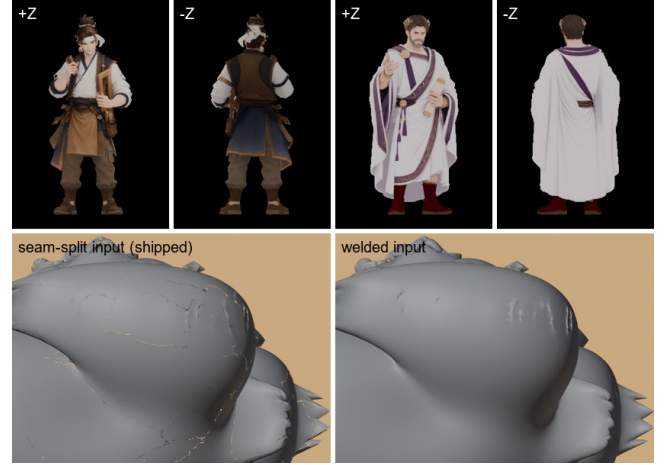
glTF units are metres [30], so a soldier, a camel and a stupa all arrive about one metre long. The milestone falls 0.005 below the plan’s 0.9 bound, so H1 as written is not met. The origin sits on the base of the bounding box in every case and within 0.44–0.58 of its width and 0.30–0.69 of its depth.

Rendered in glTF coordinates, all 15 NPCs show their faces from +Z, the front the glTF specification prescribes (Figure 1, top). They faced away from the player in the game because UniGLTF’s default conversion reverses Z, and the game’s NPC code turns an object’s +Z toward the player. The 180° turn in the import tool compensated for the importer, not for the generator. Real-world size, by contrast, cannot be recovered from the file: it has to come from the design.

## 5.2 Hygiene: closed and manifold; UV layouts like purchased meshes

H2 is not supported. After welding, all 24 raw assets are closed, edge-manifold and vertex-manifold, with no degenerate or duplicate triangles; 19 are one connected component and five have two to four. Every one has self-intersecting triangles (11–524, median 64). Closed, manifold output is what isosurface extraction produces [27], so in hindsight H2 was the wrong expectation. Other studies describe generated meshes, including Hunyuan3D 2.0 output, as noisy or non-manifold [3, 37]; we measure after welding, which may explain part of the difference.

Nor do the UV layouts stand out. The raw assets have 703–3,088 UV charts (14–63 per thousand triangles, median 32), their charts cover 48–68 % of the texture square (median 53 %), and seams duplicate 1.31–1.81 vertices per welded vertex. Our chart counts sit in the range PartUV reports for xatlas on generated meshes [37]. The game’s 81 non-generated meshes span similar ranges: the licensed player character has 10 charts per thousand triangles (its charts are stacked, so coverage is not comparable); the purchased Temple of Nike pack has 10–44 charts per thousand and covers 36–99 %;



**Figure 1: Top: two raw NPCs rendered in glTF coordinates from +Z and -Z; the faces are on the +Z side, as glTF specifies. Bottom: close-up of the companion at the shipped 82,829 triangles. Simplified from the seam-split buffer (as shipped) the surface is cracked and the orange background shows through; simplified after welding it is intact.**

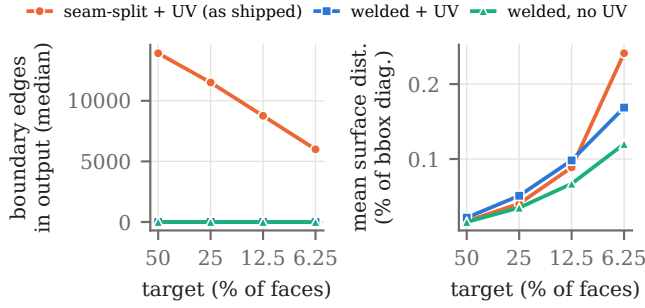
the Altar Ruins pack covers a median of 14 %; the Egypt pack maps most surfaces to a colour palette and covers almost nothing. Self-intersections are common in the non-generated meshes too (54 of 81), and half of them are not closed (41 of 81). By these measures the generated meshes are as tidy as what the game bought.

The one generated mesh that fails every check is the one we processed. The raw companion is closed, manifold and one component; the shipped version has 44,316 boundary edges, 1,442 non-manifold vertices, 2,433 components and 9,868 self-intersecting triangles (19 before). Section 5.4 shows why.

## 5.3 Cost: one uncompressed map per asset, 70 % of the build’s textures

H3 holds for all 24 assets: geometry occupies 1.28–1.60 MiB as stored, while three 4096<sup>2</sup> maps would take 256 MiB uncompressed, 64 MiB in BC7 and 32 MiB in BC1 with mips, 20–25 times the geometry even in the smallest format [1, 22]. It fails for the companion’s high-setting original (497,850 triangles), where the BC7 ratio is 4.3.

The shipped build shows what the pipeline actually did with those maps. Each of the 24 assets kept one texture, its base colour: the import tool’s material conversion copied the main texture and left the normal and metallic-roughness slots empty. That one map is stored at 4096<sup>2</sup> as uncompressed ARGB32 with a full mip chain, 85.3 MiB, because UniGLTF creates embedded textures with LoadImage rather than through Unity’s texture importer; 21 of the 24 are also flagged CPU-readable, which keeps a second copy in system memory. Together the 24 maps are 2,048 MiB, 70 % of the 2,944 MiB of texture data in the build; the largest texture from any other source is 21.3 MiB (a 4096<sup>2</sup> BC7 map). The boss’s and companion’s textures, which did go through the texture importer, were stored block-compressed (DXT1 for colour, DXT5 for normals); at DXT1 a 4096<sup>2</sup> base map would be 10.7 MiB, at BC7 21.3 MiB. An



**Figure 2: Simplifying the 24 raw assets (medians). Left: boundary edges in the output, counted after welding identical positions; the inputs have none. Seam-split input cracks at every target; welded input, with or without UVs, never does (the two lines coincide at zero). Right: mean surface distance to the original.**

**Table 3: Simplification at 6.25 % of the triangles (24 assets, up to three passes). Distance: mean surface distance, % of the bounding-box diagonal, median [max]. Colour: base-colour RMSE above the unsimplified noise floor (0–255 scale), median [max]. The maxima for W both come from the asset that stalled. \*Normal preservation on; with it off, as for S and W, all 24 also reach the target with no cracks.**

| Input                 | Reached | Cracked | Distance    | Colour     |
|-----------------------|---------|---------|-------------|------------|
| S: seam-split + UV    | 18      | 24      | 0.24 [0.32] | 3.5 [9.6]  |
| S, boundary preserved | 0       | 0       | –           | 2.4 [16.7] |
| W: welded + UV        | 23      | 0       | 0.17 [1.40] | 3.7 [37.1] |
| N: welded, no UV*     | 24      | 0       | 0.12 [0.20] | –          |

AI review of the project in July flagged the uncompressed maps; nothing was changed (Section 5.8). The dropped maps and the CPU copies were never mentioned.

#### 5.4 Simplification: the input decides, not the UVs

H4 is not supported. Welded and with UVs preserved, 23 of 24 assets reached 6.25 % of their triangles (about 3,100) in one pass with no cracks (Table 3). The exception has the most UV charts (3,088) and stalled at 3,800–3,900 triangles after three passes (two runs). Charts constrain where the simplifier may collapse, but at these budgets they rarely stop it.

What decided the outcome was the input. MeshLab’s glTF loader returns the file’s seam-split buffer, in which every UV seam already separates the surface into distinct vertices, as glTF requires [30]. MeshLab’s texture-aware simplification then treats each seam as an open boundary. With boundary preservation off (the default) the two sides of every seam move independently: every asset cracked at every target, with a median of 13,924 open edges already at 50 % (Figure 2), and six assets did not reach 6.25 % within three passes. With boundary preservation on, nothing cracks but nothing gets far: no asset went below 13,238 triangles, and the median stopped

at 21,939, about 44 % of the original. The filter reports no error in either case. The failure depends on the tool. meshoptimizer, which infers seams from matching positions [15], did not crack the same seam-split inputs but stalled: two of 24 reached 6.25 % (median 5,900 triangles), against 24 of 24 on welded input.

Welding identical positions first removes the problem in both tools; in PyMeshLab this is one call, `meshing_remove_duplicate_vertices`, which keeps UVs per corner. The cost in colour is modest: median base-colour error above the noise floor is 1.5 levels at 12.5 % (0.1 for seam-split input) and 3.7 at 6.25 % (3.5), on a 0–255 scale, with one outlier at 37 for the asset that stalled. Mean surface distance is slightly higher than for seam-split input at 50–12.5 % and lower at 6.25 %. Attribute discontinuities are a classic concern in simplification [13, 14], and the weld-first rule is documented [15]; what this case shows is how silently a common scripted path runs into them.

This is what happened to the companion. Our pipeline loaded the 497,850-triangle GLB directly and simplified it in three passes to 82,829 triangles, giving the torn mesh of Section 5.2 (Figure 1, bottom). The design note for that step says the simplifier stopped near 106k triangles because the UV islands were a hard floor. That figure came from an earlier trial run; the shipped script multiplies the target by 0.55 per pass and simply ran its three passes ( $497,850 \times 0.55^3 \approx 82,830$ ). On the welded input the same filter takes the companion to 31,116 triangles (6.25 %) in one pass with no boundary edges. The torn mesh shipped after a play check recorded only as “Play-verified” in the commit history.

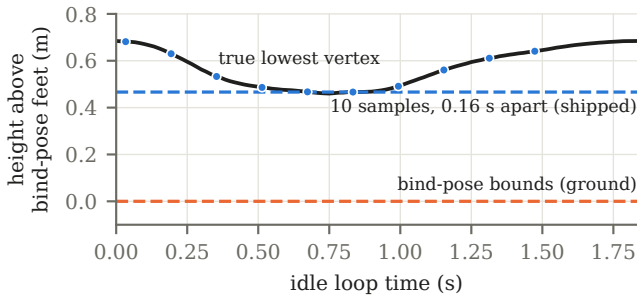
#### 5.5 History: few fixes, and most of them ours

Of 484 non-merge commits, 156 matched the pattern. After review they are 61 LEVEL, 78 OTHER, 7 ENGINE and 10 ASSET. The pattern was written around asset terms, so the comparison is within matched commits only. Seven of the ten ASSET commits are tooling written ahead of need: the NPC import tool (scale to 1.7 m, turn 180°, convert material), its application to each era’s models, the boss bake tool, and the companion’s simplification and pivot setup. Three are fixes: the boss hovering above its platform (Section 5.6), and twice the companion lying on its back after our GLB-to-Obj conversion dropped the root rotation node.

The first labelling had six ASSET fixes and found H5 not supported (three of six countable). Review moved the NPC facing fix to ENGINE (Section 5.1) and two ambiguous fixes to OTHER, as the plan’s rule requires: a double 180° turn in our own boss code, and a scale calibration whose cause mixed the rig and the engine. The boss-hover fix was also labelled with low confidence, but its cause is not in doubt (E1 reproduces it offline from the asset alone), so it stays ASSET. Two of the three remaining fixes (the lost rotation node) concern a property counting would show, and so do nine of the ten ASSET commits, so H5 is supported; because the relabelling came after the result, we treat this as weak evidence. Two of the three fixes repaired a step we had added.

#### 5.6 Grounding: the bind pose misplaces an animated character

The boss’s studio mesh is skinned with its bind-pose feet at the origin, but its idle animation lifts the whole body (Figure 3). The



**Figure 3: Boss idle loop (1.85 s) at game scale. Placing the bind-pose feet on the ground leaves the animated feet 0.46–0.68 m in the air for the whole loop. The shipped estimator (minimum of ten samples, dots) lands within about 20 mm of the loop’s minimum at any starting phase; the remaining 0.22 m is the idle bob.**

bake tool put the bind-pose feet on the platform, which leaves the animated feet 0.46–0.68 m above it on every frame. The first run-time fix read the renderer’s bounds, which in the developer’s test did not follow the pose; Unity documents skinned-mesh bounds as precomputed unless updated every frame [33]. One sampled frame can land anywhere in the 0.22 m bob. The shipped estimator, the minimum of ten samples 0.16 s apart, covers 1.44 s of the 1.85 s loop and in Blender lands within about 20 mm of the loop’s minimum at any starting phase (median 4 mm). The remaining 0.22 m is not estimator error: one constant offset cannot follow a body that moves up and down, so it is the floor for any constant, and only per-frame contact handling such as foot IK [18] goes below it. Unity’s per-frame “Feet” root mode does this for Humanoid rigs only [35]; the generated boss is imported as a Generic rig.

The constant is also calibrated on one clip. The other three animations dip lower than idle: relative to the idle-calibrated ground, the lowest frame of the attack sinks 0.27 m, the knock-down 0.08 m and the forward leap 1.19 m. In the final build these clips never play (no script fires their triggers), so this is a latent defect that wiring them up would expose. In the engine the estimator also needed an empirical 0.30 m offset, attributed in the commit record to a scale ambiguity in Unity’s BakeMesh for this rig; we did not reproduce or evaluate it.

### 5.7 The assistant’s explanations against the measurements

The two extractors found 83 distinct claims about the generated assets, 62 of them independently. Judge agreement before resolution was 76 % (Cohen’s  $\kappa = 0.64$ ); all 20 disagreements were resolved under a rule fixed in advance (a verdict needs a measurement of the asset concerned; inferences from other assets go to UNTESTABLE). Forty claims could not be tested against this study’s measurements and four concerned non-generated models. Of the 39 testable claims, 31 were confirmed and eight were contradicted or only partly supported (20.5 %), so H6, set at 20 %, is met narrowly; under the second judge’s verdicts alone the share is 31 % (16 of 51).

The pattern is clearer than the rate (Table 4). The confirmed claims are mostly numbers read off a file: triangle counts, texture sizes, bounding boxes, the root rotation. The wrong ones are mostly explanations, and the two wrong numbers were offered as explanations too: a face count at which simplification had supposedly “converged”, and a 0.04 that ruled out a cause. Five of the eight blame the generator for something the pipeline did: the importer reversed the front, our script’s round limit stopped the simplification, and the origin was where glTF puts it. The other three are a face count the shipped script never produced, a pitch with the wrong sign, and the dismissal of the idle bob. The “0.04” that ruled out the bob matches the bob in the FBX’s own units (0.041), while the hover it was compared with was in scene units, where the bob is 0.22 m; the development logs that would show which numbers were compared no longer exist, so this is our reading, not an established fact.

Wrong explanations were not corrected; they were reused. Five of the six contradicted claims were never revised (H7), and three of them, with both partly-wrong ones, are still written in the final build’s tool comments, a script and a design note. The “-Z” explanation travelled: from the NPC import tool (June 23) to the boss tool, whose comment calls it an “NPC-pipeline finding” (July 9), and to the companion’s design note as an expectation (July 15). Only one wrong claim, the companion’s pitch, was revised, after its effect (a fox standing on its head) appeared on screen five minutes later. By kind, two of five testable causal claims were wrong against three of 14 property claims and two of 14 magnitudes (H8, exploratory; the counts are too small to separate).

### 5.8 Defect lifecycles: what is seen is fixed, what is not seen ships

Table 5 is the defect record of the generated assets. H9 holds: every defect clearly visible in normal play was fixed, each within an hour of being reported, and every defect not visible on screen is in the final build. The boss’s hover took five commits in 26 minutes, and two of them introduced defects of their own (a fix that changed nothing, and one that sank the boss 1.5 m into its platform). The idle bob was reported once and accepted.

Two of the shipped defects had been seen. On July 15 an external AI review of the project, committed to the repository, estimated each of the 24 GLBs at “about 89 MB uncompressed” in the build and proposed compressing their textures; the commit that adopted the review’s fixes the same day did not adopt this one, and no later commit did. The companion’s cracks passed a play check recorded as “Play-verified”. The other clips’ predicted sinking is latent: no script in the final build fires the boss’s attack, hit or leap triggers, so only the idle loop ever plays.

## 6 Discussion

*Where the problems were.* We expected generated meshes to fail as meshes. They did not: mesh topology caused no fix, and by our measures the raw output is as closed, as manifold and as efficiently unwrapped as the meshes the game bought. The problems sat around the mesh. The generator normalises size to one metre whatever the subject. The importer reversed the front. The import path stored each asset’s texture uncompressed, mostly with a CPU copy, and dropped two of its three maps. Our simplification tore

**Table 4: The eight testable claims about the generated assets that the measurements contradict or only partly support. All were written by the AI coding assistant in commit messages, code comments or design notes during development. “Revised” = a later commit, comment or note up to the final build (87fdd16) states the corrected version; <sup>†</sup> = still written in the final build’s code or notes.**

|      | Claim (verbatim, abridged)                                                  | Verdict      | Measurement                                                                    | Revised         |
|------|-----------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------|-----------------|
| C010 | “Hunyuan front-view exports face -Z.” (tool comment, 06-23)                 | contradicted | all 15 NPC fronts at glTF +Z; the importer reverses Z                          | no <sup>†</sup> |
| C011 | historian showed its back because “Hunyuan front-view exports face -Z”      | partly       | effect real, cause is the importer                                             | no <sup>†</sup> |
| C063 | the -Z facing is “consistent across the same Hunyuan pipeline”              | partly       | consistent, but not -Z; copied into the boss tool as an “NPC-pipeline finding” | no <sup>†</sup> |
| C003 | “AI pivots are not at the feet”                                             | contradicted | origin at the base for 25/25, before and after import                          | no              |
| C029 | the idle loop’s minimum “differed ... by only 0.04 (bob was NOT the cause)” | contradicted | idle bob 0.22 m at game scale (0.041 in the FBX’s own units)                   | no              |
| C041 | simplification “converged at 105,820 faces” after three rounds              | contradicted | the shipped run stopped at 82,829 by its round limit                           | no <sup>†</sup> |
| C042 | Hunyuan’s “fragmented UV seam islands are a hard floor”, 50k unreachable    | contradicted | 31,116 triangles in one pass, welded or not                                    | no <sup>†</sup> |
| C053 | “FacePitch -90” stands the companion upright                                | contradicted | root rotation is +90°; -90 turns it head-down                                  | yes             |

**Table 5: Lifecycle of the defects involving generated assets (game history up to the final build). Channel: who first described the symptom. Hours: introduced to observed, observed to fixed. “-” = not fixed.**

| Defect                          | Channel    | Obs.  | Fix | Ships  |
|---------------------------------|------------|-------|-----|--------|
| NPCs face away                  | play-test  | 22    | 0   | no     |
| Unit scale                      | import     | 0     | 0   | no     |
| Magenta material                | import     | 22    | 0   | no     |
| Companion on its back (2 fixes) | play-test  | 0.4   | 0.1 | no     |
| Boss double turn                | play-test  | 0.3   | 0   | no     |
| Boss hover (5 fixes)            | play-test  | 52    | 0.4 | no     |
| fix 1 no-op; fix 4 sank 1.5 m   | play-test  | 0.2   | 0   | no     |
| Boss normal-map type            | assistant  | 43    | 0   | no     |
| Idle bob 0.22 m                 | play-test  | 52    | -   | yes    |
| Base maps uncompressed          | AI review  | 560   | -   | yes    |
| Normal/MR maps dropped          | this audit | 2,513 | -   | yes    |
| CPU-readable copies             | this audit | 2,513 | -   | yes    |
| Companion torn                  | this audit | 1,953 | -   | yes    |
| Other clips sink (latent)       | this audit | -     | -   | latent |

the companion. An animation lifted the boss off the ground. Of these, only the size normalisation and the animation belong to the generator; the rest belong to the pipeline built around it, most of it written by the assistant.

*Explanations are where the assistant went wrong.* The assistant was mostly right when it read numbers off files and less reliable when it explained them. Most of its wrong explanations had a direction: they named the newest and most conspicuous component, the generator, as the cause of what an older component, the importer or our own script, had done. They also had a life. A causal sentence in a commit message becomes a comment above the line it justifies (`_faceYaw = 180; // Hunyuan front-view exports face -Z`); the comment is then reused as a fact and copied into the next tool. Nothing in this workflow puts a written explanation back to the

test once the symptom goes away. The one wrong claim that was revised was revised because its symptom stayed on screen. That fixes can carry new defects is well known [42]; here the more lasting product of a fix was its explanation. A cheap safeguard follows: when an assistant writes a cause into a comment or a design note, attach the check that would confirm it, or mark it as a hypothesis.

*Play-testing sees only what is on screen.* Every defect that a player could see was found and fixed, mostly within the hour. Everything else shipped: an uncompressed texture costs memory, not pixels; a missing normal map looks like a stylistic choice; a hairline crack on a small companion passes a play check. One of these had even been flagged by an AI review and was deferred without a recorded reason. For assistant-built pipelines, where the developer’s main check is to run the game and look, the gap between what is visible and what ships is where a pre-import or pre-build check earns its keep.

*A pre-import checklist.* Derived from this case and not validated on other projects.

- (1) *Size and front.* Treat the largest extent as unknown and supply real size from the design. Render one view from +Z before import and one after, to separate the asset’s front from the importer’s axis convention.
- (2) *What the engine actually stores.* Read texture format, size, readability and which maps the material references from the build, not from the source file; an uncompressed 4096<sup>2</sup> map is four times its BC7 size, and a dropped normal map is invisible to a memory check.
- (3) *Simplification input.* Weld before simplifying and count boundary edges after; a seam-split buffer simplifies without any error message, and in MeshLab usually returns the requested triangle count.
- (4) *Animated contact.* For rigged assets, evaluate every clip offline and compare its lowest vertex with the bind pose and with the clip used for placement.

- (5) *Explanations*. Treat a cause written by the assistant as a hypothesis until a measurement backs it, and keep that measurement next to the comment.

Items 1 and 3 restate existing guidance on real-world scale and origin [16] and on welding before simplification [15], and texture memory is a familiar concern [32]; what this case adds is where the violations came from, which ones went unnoticed, and how their explanations went wrong.

## 7 Limitations

*Scope*. One generator and version, one importer, one game: 24 meshes from one setting and two from other routes. Hunyuan 3D’s service is closed and changes over time. The 50k setting and 4096<sup>2</sup> textures were our defaults, not a sweep. The texture findings depend on UniGLTF and our material tool; another importer would store the maps differently. We report counts, not population estimates.

*No frame-time measurements*. Memory figures are texel counts and stored sizes from the build, not profiler readings; we did not measure draw calls, frame time or GPU residency per asset.

*Not blind*. The author built the game, wrote the commits B5 labels and wrote the plan. The labels were drafted by two AI labellers and reviewed by the AI assistant and the author; there was no independent human rater. Commit messages record what the developer believed at the time. Keyword search misses commits that never used the words.

*Grounding*. Ground truth comes from Blender’s evaluation of the FBX, not from the engine; the in-engine offset and the predicted sinking in the other clips were not checked in play. The boss is one rig.

*Claims and lifecycles*. The claims are what the assistant wrote and the developer accepted, not what it would say if asked; the conversations of the development period no longer exist, so we cannot tell which claims the developer challenged, and effort is counted in commits and hours, not turns. Extraction and judging were done by AI agents with a written rule and author sign-off; the two judges agreed on 76 % of claims before resolution, and the wrong-claim share is 20 % under the resolved verdicts and 31 % under the second judge’s alone. Nearly half of the claims could not be tested, mostly those about engine-side readings we did not reproduce. One assistant, one developer and one game.

*Appearance*. Visual fidelity to the input image was not scored. The colour error in B4 uses the nearest original vertex and only the base colour map.

*Authorship*. The game, this audit and this text were made with an AI coding assistant. Plan, scripts, outputs, labels and the review reports that prompted the revisions are public so that the numbers can be checked without trusting the author or the assistant.

## 8 Deviations from the Plan

- B4’s with-UV condition was split into seam-split (S) and welded (W) inputs after B2 found the shipped companion torn.

- After an internal fact-check and review of the first draft, we added: the facing renders (B1), the non-generated baseline (B2), the shipped-build texture readout (B3), S with boundary preservation, a matched no-UV control and a colour error (B4), all four clips and all starting phases (E1). A scene-memory comparison in the first draft was dropped: its premise (compressed textures) was wrong.
- The first draft stated that UV charts do not overlap; the rasterised coverage cannot show that, and the claim was removed.
- B5 labels were drafted by two labellers rather than one and reviewed twice. The second review moved three fixes out of ASSET, which changed H5’s outcome (Section 5.5). The reviewer added an *origin* field and the fix/feature split, neither of which was in the plan.
- C and D follow a second plan (63b4a34) written after the first results were known and before any claim was extracted; the judges’ resolution rule was written before resolving. Two measurements were added for judging claims, both listed in that plan in advance.
- The plan’s B3 metric was bytes as uploaded; we report bytes as stored in the file, then what the build stored.
- E1 did not evaluate the shipped constant offset (0.30 m), which corrects an engine-side reading we did not reproduce. The Wilson interval planned for H2 was dropped: the 24 assets are not a random sample. Output file names differ from those listed in the plan.
- A second internal fact-check of the revised draft led to the meshoptimizer comparison and to corrections of several numbers and phrasings; both review rounds are in the repository.

## Acknowledgments

The game, its import tools and this audit were built with the help of an AI coding assistant (Anthropic’s Claude in Claude Code), which also drafted the commit labels, the analysis scripts and parts of this text, and ran the internal fact-check and review that led to the revised version. The author reviewed the labels and the final text. Assets were generated with Tencent’s Hunyuan 3D service. No external funding.

## References

- [1] Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michal Iwanicki, and Sébastien Hillaire. 2018. *Real-Time Rendering* (4th ed.). A K Peters/CRC Press, Boca Raton, FL. doi:10.1201/9781315365459
- [2] Andrew Begemann and James Hutson. 2026. Prompted Props, Human Pipelines: Evaluating AI-Generated 3D Assets for Game-Ready Environments. *International Journal of Human Research and Social Science Studies* 3, 6 (June 2026), 647–661. doi:10.5281/zenodo.21064847
- [3] Kunal Bhosikar, Preet Savalia, Lokender Tiwari, and Brojeshwar Bhowmick. 2026. Fast and Robust Mesh Simplification for Generated and Real-World 3D Assets. arXiv:2605.14029 [cs.GR] <https://arxiv.org/abs/2605.14029>
- [4] Mark Boss, Zixuan Huang, Aaryaman Vasishtha, and Varun Jampani. 2025. SF3D: Stable Fast 3D Mesh Reconstruction with UV-unwrapping and Illumination Disentanglement. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 16240–16250. arXiv:2408.00653. [https://openaccess.thecvf.com/content/CVPR2025/html/Boss\\_SF3D\\_Stable\\_Fast\\_3D\\_Mesh\\_Reconstruction\\_with\\_UV-unwrapping\\_and\\_Illumination\\_CVPR\\_2025\\_paper.html](https://openaccess.thecvf.com/content/CVPR2025/html/Boss_SF3D_Stable_Fast_3D_Mesh_Reconstruction_with_UV-unwrapping_and_Illumination_CVPR_2025_paper.html)
- [5] Yiwen Chen, Tong He, Di Huang, Weicai Ye, Sijin Chen, Jiaxiang Tang, Xin Chen, Zhongang Cai, Lei Yang, Gang Yu, Guosheng Lin, and Chi Zhang. 2025. MeshAnything: Artist-Created Mesh Generation with Autoregressive

- Transformers. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.10163. [https://proceedings.iclr.cc/paper\\_files/paper/2025/hash/7fc69f53a6ca2664ef23a9597903ca6f-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2025/hash/7fc69f53a6ca2664ef23a9597903ca6f-Abstract-Conference.html)
- [6] Paolo Cignoni, Marco Callieri, Massimiliano Corsini, Matteo Dellepiane, Fabio Ganovelli, and Guido Ranzuglia. 2008. MeshLab: an Open-Source Mesh Processing Tool. In *Eurographics Italian Chapter Conference*. The Eurographics Association, 129–136. doi:10.2312/LocalChapterEvents/ItalChap/ItalianChapConf2008/129-136
  - [7] Paolo Cignoni, Claudio Rocchini, and Roberto Scopigno. 1998. Metro: Measuring Error on Simplified Surfaces. *Computer Graphics Forum* 17, 2 (1998), 167–174. doi:10.1111/1467-8659.00236
  - [8] Bryant Francis. 2024. How Devs Can Spot AI-Generated 3D Models. Game Developer. Published 2024-11-05; reports observations by 3D artist Liz Edwards. Accessed 2026-10-05. <https://www.gamedeveloper.com/art/how-devs-can-spot-ai-generated-3d-models>
  - [9] Galashots. 2026. decimate\_gear v2: Weld Seams, Decimate, Fresh UVs, Bake from Source (Fixes Seam Cracks). GitHub Pull Request #207, galaquest-public. Merged 2026-09-23. Accessed 2026-10-05. <https://github.com/Galashots/galaquest-public/pull/207>
  - [10] Michael Garland and Paul S. Heckbert. 1997. Surface Simplification Using Quadric Error Metrics. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '97)*. ACM Press/Addison-Wesley, 209–216. doi:10.1145/258734.258849
  - [11] Michael Garland and Paul S. Heckbert. 1998. Simplifying Surfaces with Color and Texture Using Quadric Error Metrics. In *Proceedings Visualization '98*. IEEE, 263–269. doi:10.1109/VISUAL.1998.745312
  - [12] Yuze He, Yushi Bai, Matthieu Lin, Wang Zhao, Yubin Hu, Jenny Sheng, Ran Yi, Juanzi Li, and Yong-Jin Liu. 2023. T<sup>3</sup>Bench: Benchmarking Current Progress in Text-to-3D Generation. arXiv:2310.02977 [cs.CV] doi:10.48550/arXiv.2310.02977
  - [13] Hugues Hoppe. 1996. Progressive Meshes. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*. ACM, 99–108. doi:10.1145/237170.237216
  - [14] Hugues Hoppe. 1999. New Quadric Metric for Simplifying Meshes with Appearance Attributes. In *Proceedings Visualization '99*. IEEE, 59–66. doi:10.1109/VISUAL.1999.809869
  - [15] Arseny Kapoulkine. 2026. meshoptimizer: Mesh Optimization Library – Simplification. Accessed 2026-10-05. <https://meshoptimizer.org/>
  - [16] Khronos 3D Commerce Working Group. 2025. Real-time Asset Creation Guidelines, Version 2.0.0. The Khronos Group, CC BY 4.0. Last updated August 2025; accessed 2026-10-05. <https://github.com/KhronosGroup/3DC-Asset-Creation/blob/main/asset-creation-guidelines/RealtimeAssetCreationGuidelines.md>
  - [17] Khronos Group. 2026. glTF 2.0 Blender Importer/Exporter Documentation: Import Options, “Merge Vertices”. glTF-Blender-IO repository, docs/blender\_docs/scene\_gltf2.rst. Accessed 2026-10-05. [https://github.com/KhronosGroup/glTF-Blender-IO/blob/main/docs/blender\\_docs/scene\\_gltf2.rst](https://github.com/KhronosGroup/glTF-Blender-IO/blob/main/docs/blender_docs/scene_gltf2.rst)
  - [18] Lucas Kovar, John Schreiner, and Michael Gleicher. 2002. Footskate Cleanup for Motion Capture Editing. In *Proceedings of the 2002 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '02)*. ACM, 97–104. doi:10.1145/545261.545277
  - [19] Biwen Lei, Yang Li, Xinhai Liu, Shuhui Yang, Lixin Xu, Jingwei Huang, Ruining Tang, Haohan Weng, Jian Liu, Jing Xu, et al. 2025. Hunyuan3D Studio: End-to-End AI Pipeline for Game-Ready 3D Asset Generation. arXiv:2509.12815 [cs.CV] doi:10.48550/arXiv.2509.12815
  - [20] Hsueh-Ti Derek Liu, Xiaoting Zhang, and Cem Yuksel. 2025. Simplifying Textured Triangle Meshes in the Wild. *ACM Transactions on Graphics* 44, 6 (2025), 1–16. SIGGRAPH Asia 2025; arXiv:2409.15458. doi:10.1145/3763277
  - [21] Songrun Liu, Zachary Ferguson, Alec Jacobson, and Yotam Gingold. 2017. Seamless: Seam Erasure and Seam-Aware Decoupling of Shape from Mesh Resolution. *ACM Transactions on Graphics* 36, 6 (2017), 1–15. doi:10.1145/3130800.3130897
  - [22] Microsoft. 2026. BC7 Format (Direct3D 11). Microsoft Learn, Win32 apps documentation. Accessed 2026-10-05. <https://learn.microsoft.com/en-us/windows/win32/direct3d11/bc7-format>
  - [23] Alessandro Muntoni and Paolo Cignoni. 2021. PyMeshLab. Concept DOI; version 2025.7.post1 used in this study. doi:10.5281/zenodo.4438750
  - [24] Ruchi Panchanadikar and Guo Freeman. 2024. “I’m a Solo Developer but AI is My New Ill-Informed Co-Worker”: Envisioning and Designing Generative AI to Support Indie Game Development. *Proceedings of the ACM on Human-Computer Interaction* 8, CHI PLAY, Article 317 (2024), 26 pages. doi:10.1145/3677082
  - [25] Pedro V. Sander, John Snyder, Steven J. Gortler, and Hugues Hoppe. 2001. Texture Mapping Progressive Meshes. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*. ACM, 409–416. doi:10.1145/383259.383307
  - [26] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. 2024. MeshGPT: Generating Triangle Meshes with Decoder-Only Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 19615–19625. arXiv:2311.15475. [https://openaccess.thecvf.com/content/CVPR2024/html/Siddiqui\\_MeshGPT\\_Generating\\_Triangle\\_Meshes\\_with\\_Decoder-Only\\_Transformers\\_CVPR\\_2024\\_paper.html](https://openaccess.thecvf.com/content/CVPR2024/html/Siddiqui_MeshGPT_Generating_Triangle_Meshes_with_Decoder-Only_Transformers_CVPR_2024_paper.html)
  - [27] Team Hunyuan3D, Shuhui Yang, Mingxin Yang, Yifei Feng, Xin Huang, Sheng Zhang, Zebin He, Di Luo, Haolin Liu, Yunfei Zhao, et al. 2025. Hunyuan3D 2.1: From Images to High-Fidelity 3D Assets with Production-Ready PBR Material. arXiv:2506.15442 [cs.CV] doi:10.48550/arXiv.2506.15442
  - [28] Tencent Cloud. 2026. Submit Hunyuan 3D Professional Job (SubmitHun-yuanTo3DProJob) – Tencent Hunyuan 3D API Documentation. Official API documentation (in Chinese), last updated 2026-05-11. Accessed 2026-10-05. <https://cloud.tencent.com/document/product/1804/123447>
  - [29] Tencent Hunyuan3D Team. 2025. Hunyuan3D-2.1 Source Code (textureGen-Pipeline.py, utils/uvwrap\_utils.py, hy3dshape/postprocessors.py). Main branch read 2026-10-05. <https://github.com/Tencent-Hunyuan/Hunyuan3D-2.1>
  - [30] The Khronos 3D Formats Working Group. 2021. glTF 2.0 Specification. Specification Version 2.0.1. The Khronos Group Inc. <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>
  - [31] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. 2024. TriposR: Fast 3D Object Reconstruction from a Single Image. arXiv:2403.02151 [cs.CV] doi:10.48550/arXiv.2403.02151
  - [32] Unity Technologies. 2026. Art Optimization Tips for Mobile Game Developers, Part 1. Accessed 2026-10-05. <https://unity.com/how-to/mobile-game-optimization-tips-part-1>
  - [33] Unity Technologies. 2026. Renderer.localBounds. Unity Scripting API. Accessed 2026-10-05. <https://docs.unity3d.com/ScriptReference/Renderer-localBounds.html>
  - [34] Unity Technologies. 2026. SkinnedMeshRenderer.BakeMesh. Unity Scripting API. Accessed 2026-10-05. <https://docs.unity3d.com/ScriptReference/SkinnedMeshRenderer.BakeMesh.html>
  - [35] Unity Technologies. 2026. Unity Manual: Root Motion – How It Works. Accessed 2026-10-05. <https://docs.unity3d.com/Manual/RootMotion.html>
  - [36] Visual Computing Lab, ISTI-CNR. 2026. PyMeshLab Documentation: List of Filters. Entries meshing\_decimation\_quadric\_edge\_collapse\_with\_texture, compute\_texcoord\_transfer\_vertex\_to\_wedge, meshing\_merge\_close\_vertices. Accessed 2026-10-05. [https://pymeshlab.readthedocs.io/en/latest/filter\\_list.html](https://pymeshlab.readthedocs.io/en/latest/filter_list.html)
  - [37] Zhaoning Wang, Xinyue Wei, Ruoxi Shi, Xiaoshuai Zhang, Hao Su, and Minghua Liu. 2025. PartUV: Part-Based UV Unwrapping of 3D Meshes. In *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*. ACM, 1–12. arXiv:2511.16659. doi:10.1145/3757377.3763843
  - [38] Jiafeng Wu, Zhuofan Lou, Jian Liu, Dazhao Du, Chunchao Guo, and Song Guo. 2026. From Visual Synthesis to Interactive Worlds: Toward Production-Ready 3D Asset Generation. arXiv:2604.23629 [cs.CV] <https://arxiv.org/abs/2604.23629>
  - [39] Tong Wu, Guandao Yang, Zhibing Li, Kai Zhang, Ziwei Liu, Leonidas Guibas, Dahua Lin, and Gordon Wetzstein. 2024. GPT-4V(ision) is a Human-Aligned Evaluator for Text-to-3D Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 22227–22238. arXiv:2401.04092. [https://openaccess.thecvf.com/content/CVPR2024/html/Wu\\_GPT-4Vision\\_is\\_a\\_Human-Aligned\\_Evaluator\\_for\\_Text-to-3D\\_Generation\\_CVPR\\_2024\\_paper.html](https://openaccess.thecvf.com/content/CVPR2024/html/Wu_GPT-4Vision_is_a_Human-Aligned_Evaluator_for_Text-to-3D_Generation_CVPR_2024_paper.html)
  - [40] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. 2025. Structured 3D Latents for Scalable and Versatile 3D Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 21469–21480. arXiv:2412.01506. [https://openaccess.thecvf.com/content/CVPR2025/html/Xiang\\_Structured\\_3D\\_Latents\\_for\\_Scalable\\_and\\_Versatile\\_3D\\_Generation\\_CVPR\\_2025\\_paper.html](https://openaccess.thecvf.com/content/CVPR2025/html/Xiang_Structured_3D_Latents_for_Scalable_and_Versatile_3D_Generation_CVPR_2025_paper.html)
  - [41] Jiale Xu, Weihao Cheng, Yiming Gao, Xintao Wang, Shenghua Gao, and Ying Shan. 2024. InstantMesh: Efficient 3D Mesh Generation from a Single Image with Sparse-view Large Reconstruction Models. arXiv:2404.07191 [cs.CV] doi:10.48550/arXiv.2404.07191
  - [42] Zuoning Yin, Ding Yuan, Yuanqian Zhou, Shankar Pasupathy, and Lakshmi Bairavasundaram. 2011. How Do Fixes Become Bugs?. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE '11)*. ACM, 26–36. doi:10.1145/2025113.2025121
  - [43] Yuhang Zhang, Mengchen Zhang, Tong Wu, Tengfei Wang, Gordon Wetzstein, Dahua Lin, and Ziwei Liu. 2025. 3DGen-Bench: Comprehensive Benchmark Suite for 3D Generative Models. arXiv:2503.21745 [cs.CV] doi:10.48550/arXiv.2503.21745
  - [44] Penggan Zhao. 2026. Counting What Inspection Misses: Auditing an LLM-Generated Quiz Bank for an Educational Game. Preprint. doi:10.5281/zenodo.23150165
  - [45] Penggan Zhao. 2026. Feature or Leak? Guarding an LLM-Driven NPC Against Answer Leakage in an Educational Game. Preprint. doi:10.5281/zenodo.23150869
  - [46] Zibo Zhao, Zeqiang Lai, Qingxiang Lin, Yunfei Zhao, Haolin Liu, Shuhui Yang, Yifei Feng, Mingxin Yang, Sheng Zhang, Xianghui Yang, et al. 2025. Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation. arXiv:2501.12202 [cs.CV] doi:10.48550/arXiv.2501.12202